

Exploratory Data Mining of 240,000 Code Snippets: Empirical Patterns, Structural Formatting, and Model Fingerprints in Human vs. AI Code Synthesis

Hassan Elkady — Computer Engineering Student, Arab Academy for Science, Technology and Maritime Transport (AAST)

August 2026 | Zenodo Dataset (DOI: 10.5281/zenodo.15423067): 60,000 Problem Tasks / 240,000 Programs Evaluated

Abstract

As Large Language Model (LLM) code generators become integral to software development, understanding their structural, visual, and syntactical tendencies is critical for automated code analysis, review, and AI detection. Rather than starting with a preconceived hypothesis, this paper presents a purely exploratory, data-driven investigation analyzing **240,000 code snippets** across **60,000 problem tasks** (30,000 Python and 30,000 Java tasks) from the Zenodo Large-Scale Dataset (10.5281/zenodo.15423067). Each task provides four parallel implementations authored by senior human developers and three major AI model families: OpenAI ChatGPT, DeepSeek-Coder, and Alibaba Qwen-Coder.

Glossary of Technical & Statistical Terms for General Readers

- **Lines of Code (LOC):** The count of non-blank, executable, or structural lines of code.
- **Vertical Whitespace (%):** The percentage of physical lines in a snippet that are empty/blank lines.
- **Comment Density (%):** The percentage of non-blank lines that contain comments or docstrings.
- **Mann-Whitney U Test:** A statistical test comparing two groups without assuming a bell-curve distribution.
- **Holm-Bonferroni FWER Correction (p_{adj}):** A procedure adjusting p-values to control false discovery rates.
- **Kruskal-Wallis H-Test:** A statistical test evaluating whether three or more independent groups differ significantly.

2. Quantitative Results & Empirical Distributions

2.1 Python Sub-Dataset Analysis (30,000 Tasks / 120,000 Code Snippets)

Author / Model Family	Lines of Code (LOC)	Comment Density (%)	Vertical Whitespace (%)	Mean Line Length (chars)	Docstring Rate (%)
Human Developer	14.47 ± 18.20 [Med: 9.0]	4.69% ± 9.29%	0.32% ± 1.85%	42.83 ± 14.12	3.0%
OpenAI ChatGPT	9.64 ± 6.12 [Med: 8.0]	8.68% ± 11.20%	19.97% ± 8.42%	37.81 ± 10.55	19.0%
DeepSeek-Coder	11.45 ± 7.10 [Med: 11.0]	17.95% ± 12.85%	14.71% ± 7.90%	37.03 ± 9.88	55.0%
Alibaba Qwen-Coder	12.02 ± 11.85 [Med: 9.0]	4.63% ± 10.50%	4.27% ± 5.12%	39.75 ± 12.01	26.0%

2.2 Java Sub-Dataset Analysis (30,000 Tasks / 120,000 Code Snippets)

Author / Model Family	Lines of Code (LOC)	Comment Density (%)	Vertical Whitespace (%)	Mean Line Length (chars)	Docstring Rate (%)
Human Developer	14.69 ± 19.50 [Med: 10.0]	0.00% ± 0.22%	3.43% ± 4.50%	40.41 ± 12.85	0.0%
OpenAI ChatGPT	11.49 ± 8.25 [Med: 9.0]	6.98% ± 10.15%	16.08% ± 7.10%	37.75 ± 9.90	1.0%
DeepSeek-Coder	13.86 ± 8.92 [Med: 13.0]	8.53% ± 11.40%	12.56% ± 6.85%	35.51 ± 8.75	2.0%
Alibaba Qwen-Coder	10.54 ± 9.15 [Med: 7.0]	17.14% ± 15.20%	3.26% ± 4.10%	37.16 ± 10.12	0.0%

Figure 1: Exploratory Pattern Mining Across 240,000 Code Snippets

3. Unbiased Pattern Discoveries & Discussion

1. Single-Function Compactness: Single-function problem tasks show that AI models produce concise code (9.64 - 13.86 LOC) compared to human solutions (14.47 - 14.69 LOC).

2. Model Family Fingerprints: DeepSeek-Coder incorporates docstrings in 55.0% of Python functions; ChatGPT allocates 19.97% of lines to vertical whitespace; Qwen-Coder exhibits dense single-line commenting in Java (17.14%).

4. Conclusion & References

This hypothesis-free study of 240,000 code snippets demonstrates that LLM code generation exhibits distinct, model-specific visual formatting signatures while synthesizing concise single-function routines.